Практическое занятие 2 "Проектирование и реализация REST API"

Практическое занятие №2. Проектирование и реализация REST API

Ветка репозитория: `pz22`

---

1. ЦЕЛЬ РАБОТЫ

Освоение принципов проектирования RESTful API, реализация эндпоинтов для CRUD-операций, документирование API (Swagger/OpenAPI или ручное описание).

---

2. НЕОБХОДИМЫЕ ЗНАНИЯ

- Принципы REST (ресурсы, методы, идемпотентность, коды состояния).
- Структура URL для REST API.
- Статус коды HTTP (200, 201, 204, 400, 401, 404, 500).
- Инструменты документирования API (Swagger UI).

---

3. ЗАДАЧИ РАБОТЫ (индивидуально)

- Спроектировать REST API для заданной предметной области (минимум 3 ресурса).
- Реализовать все необходимые эндпоинты (CRUD + дополнительные операции по варианту).
- Добавить валидацию входных данных.
- Задокументировать API (описать эндпоинты, примеры запросов/ответов). Документация может быть в виде отдельного файла (`API.md`) или через Swagger/OpenAPI.

- Протестировать API с помощью Postman или аналогичного инструмента.

---

4. ПОРЯДОК ВЫПОЛНЕНИЯ (общий для всех вариантов)

Часть 1. Проектирование API

- Определите ресурсы (сущности) для вашей предметной области (минимум 3).
- Для каждого ресурса определите операции:
  - `GET /resource` – список
  - `GET /resource/:id` – детали
  - `POST /resource` – создание
  - `PUT /resource/:id` – полное обновление
  - `PATCH /resource/:id` – частичное обновление
  - `DELETE /resource/:id` – удаление
- Определите коды ответов и возможные ошибки.

Часть 2. Реализация (Node.js/Express)

- Создайте роуты и контроллеры для каждого ресурса.
- Для хранения данных используйте **временное хранилище в массиве** (в реальном проекте позже замените на БД).
- Реализуйте валидацию входных данных (проверка обязательных полей, типов).

Часть 3. Документирование API

- Создайте файл `API.md`, в котором опишите все эндпоинты (метод, URL, параметры, пример тела запроса, пример ответа, возможные ошибки).
- (Опционально) Установите `swagger-jsdoc` и `swagger-ui-express` для автоматической документации.

Часть 4. Тестирование

- Запустите сервер (`npm run dev`).

- Используйте Postman или Insomnia для проверки каждого эндпоинта.

---

5. СОДЕРЖАНИЕ ОТЧЕТА

- Титульный лист, цель работы.
- Таблица с эндпоинтами (метод, URL, описание, пример тела запроса, ожидаемый ответ).
- Скриншоты документации API (Swagger UI или фрагменты `API.md`).
- Скриншоты запросов в Postman (не менее 5 различных операций).
- Ответы на контрольные вопросы.
- Выводы.

---

6. ТРЕБОВАНИЯ К СДАЧЕ

- Отчёт (PDF) загружается в СЭО.
- Ссылка на репозиторий GitHub с кодом (ветка **`pz22`**).
- **Индивидуальное выполнение. Вариант** **Темы** определяется по **Генератору вариантов.**

---

7. КОНТРОЛЬНЫЕ ВОПРОСЫ (30 вопросов)

1. Какие методы HTTP считаются идемпотентными? Почему?
2. Какой статус код возвращается при успешном создании ресурса (POST)? При успешном удалении (DELETE)?
3. Как передать параметры фильтрации в GET-запросе? Приведите пример URL.
4. В чём разница между методами PUT и PATCH? Когда что использовать?
5. Как задокументировать API с помощью OpenAPI (Swagger)? Какие инструменты используются?
6. Что такое REST? Назовите основные принципы REST архитектуры.
7. Какой статус код следует возвращать, если ресурс не найден (GET /items/999)?

8. Что такое «именованные ресурсы» и как их правильно именовать в URL (множественное/единственное число)?
9. Как в Express получить параметры из URL (например, `/users/:id`) и строку запроса (`?page=1`)?
10. Как в Express обработать тело запроса в формате JSON? Какой middleware для этого нужен?
11. Какой статус код возвращается при невалидных входных данных (например, отсутствие обязательного поля)?
12. Что такое HATEOAS? Обязательно ли его использовать в REST API?
13. Как спроектировать URL для вложенных ресурсов (например, комментарии к посту)? Приведите пример.
14. Какие заголовки HTTP обычно используются для кэширования в REST API?
15. Как реализовать пагинацию в REST API? Через какие параметры (page, limit, offset)?
16. Как реализовать сортировку (order by) через параметры запроса? Приведите пример.
17. Что такое версионирование API? Какие способы версионирования вы знаете?
18. Как реализовать поиск (search) в REST API через параметры строки запроса?
19. Как тестировать REST API с помощью Postman? Какие коллекции можно создать?
20. Что такое CORS и как настроить его для вашего API в Express?
21. Как в REST API обрабатывать ошибки (единый формат ответа с ошибкой)? Приведите пример JSON.
22. Какие типы аутентификации чаще всего используются в REST API (Basic, Bearer token, OAuth)?
23. Как в OpenAPI (Swagger) описать схему данных (модель) для ресурса?
24. Как в Express организовать валидацию входных данных с помощью библиотек (Joi, express-validator)?
25. Что такое «межресурсные связи» и как их реализовать в REST API (например, получить заказы пользователя)?

26. Какие статус коды используются для редиректов (3xx)? Нужны ли они в REST API?
27. Как в документации API указать возможные коды ошибок и их описание?
28. Что такое `idempotency key` и зачем он нужен в API?
29. Как в REST API реализовать частичное обновление ресурса (PATCH) – какие трудности?
30. Как протестировать API автоматически (например, с помощью Jest и Supertest)?

---

8. КРИТЕРИИ ОЦЕНКИ

- Полнота спроектированных эндпоинтов (минимум 3 ресурса, все CRUD-операции).
- Корректная реализация на Express.js (роуты, контроллеры, валидация).
- Использование статус кодов HTTP и обработка ошибок.
- Наличие документации API (Swagger или файл `API.md`).
- Тестирование через Postman (скриншоты в отчёте).
- Качество ответов на контрольные вопросы.
- Ссылка на репозиторий с веткой `pz22` и инструкцией по запуску.